

**Analyzing Global Technology Trends:
Insights from the Stack Overflow Developer Survey**

IBM Data Analyst Capstone Project

Ankit Ram Samant

SSN COLLEGE OF ENGINEERING

Contact: ankitransamant@gmail.com

24th October 24, 2025

Table of Contents

1. Executive Summary	3
2. Introduction	4
3. Literature Review	5
4. Methodology	6
4.1 Data Collection	
4.2 Data Cleaning and Preparation	
4.3 Tools and Technologies Used	
5. Exploratory Data Analysis (EDA)	8
5.1 Descriptive Statistics	
5.2 Trends in Programming Languages	
5.3 Database and Platform Analysis	
6. Data Visualization and Dashboard Creation	10
6.1 Current Technology Usage Dashboard	
6.2 Future Technology Trends Dashboard	
6.3 Demographics Dashboard	
7. Discussion of Insights	13
8. Conclusion and Recommendations	14
9. Acknowledgments	15
10. References	16
11. Appendices	17

Executive Summary

This report presents an analytical study of global technology trends using data from the **Stack Overflow Developer Survey**. The project was completed as part of the **IBM Data Analyst Capstone**, simulating the role of an Associate Data Analyst in a technology consulting firm. The primary objective of the study is to extract meaningful insights into the current and future use of technologies—including programming languages, databases, frameworks, and platforms—that shape the modern software development landscape.

The dataset contains responses from developers worldwide, providing a comprehensive overview of tools they currently use and technologies they aspire to learn. The analysis process involved **data collection, data wrangling, exploratory data analysis (EDA), and visualization** using industry-standard tools such as **Python (Pandas, NumPy, Matplotlib, Seaborn)** and **SQL**. Interactive dashboards were also created using **IBM Cognos Analytics** and **Google Looker Studio** to visualize key findings effectively.

The study highlights significant trends such as the dominance of **Python, JavaScript, and SQL** among current technologies and the growing interest in **Go, TypeScript, and Rust** as emerging languages. Similarly, databases like **MySQL and PostgreSQL** continue to lead in usage, while **MongoDB** shows increasing popularity among developers seeking NoSQL solutions.

Through visual storytelling and data-driven insights, the report provides a clear understanding of technology adoption patterns, helping organizations anticipate shifts in the tech landscape. These insights can guide **strategic business decisions**, workforce training, and technology investments.

The project concludes by emphasizing the importance of continuous learning, data-driven decision-making, and the value of visualization tools in communicating analytical results effectively.

Introduction

In today's data-driven world, understanding technology usage trends is essential for organizations aiming to stay competitive in the rapidly evolving digital landscape. This project, undertaken as part of the **IBM Data Analyst Capstone**, explores global technology trends using the **Stack Overflow Developer Survey dataset**. The report focuses on analyzing the technologies developers currently use and the ones they aspire to learn in the future, offering valuable insights into the direction of the tech industry.

As an **Associate Data Analyst** at a technology consulting organization, the primary objective is to analyze the data and uncover insights that support **strategic business decision-making**. The survey dataset provides a detailed snapshot of developers' preferences, experiences, and technology choices across multiple domains, including **programming languages, databases, platforms, and frameworks**.

The project adopts a systematic approach involving **data collection, cleaning, exploration, and visualization**. Using **Python, SQL, and IBM Cognos Analytics**, the data was analyzed to identify patterns and shifts in the developer community's preferences. By transforming raw data into meaningful insights and visual dashboards, the project demonstrates how analytics can drive informed business and technological decisions.

This study not only showcases the application of **data analytics methodologies** but also highlights the role of visualization in enhancing data storytelling. The findings derived from this analysis will help organizations anticipate emerging technologies, understand developer adoption patterns, and align their strategies with evolving industry trends.

Literature Review

Understanding global technology trends has been a central theme in recent analytical and consulting reports by leading firms such as **Deloitte**, **McKinsey & Company**, and **Gartner**. These studies highlight the increasing importance of **data analytics**, **artificial intelligence**, and **cloud computing** in shaping the technology landscape. Similarly, the **Stack Overflow Developer Survey** has become one of the most credible sources for identifying annual shifts in developer preferences, tools, and platforms.

Deloitte's *Tech Trends 2024* report emphasizes how organizations are leveraging emerging technologies to gain competitive advantages, with particular focus on **AI-driven automation**, **data integration**, and **cross-platform development**. McKinsey's *State of AI and Technology Forward* studies similarly stress the role of **programming versatility** and **data literacy** as critical skills for the future workforce. These insights align with the present project's objectives of identifying which technologies are most valued today and which are gaining momentum.

Several academic and industry researchers have used survey-based datasets, like Stack Overflow's, to analyze **developer behavior**, **tool adoption**, and **career trends**. Studies show consistent dominance of languages such as **Python**, **JavaScript**, and **SQL**, driven by their versatility and use in web development, data analysis, and automation. Meanwhile, emerging languages like **Go**, **Rust**, and **TypeScript** are gaining attention due to performance, scalability, and developer efficiency.

However, most existing studies focus on broad patterns without detailed visualization or dashboard-based insights. This capstone project fills that gap by offering **interactive dashboards** and **visual storytelling**, enabling more accessible interpretation of complex datasets. It integrates academic research and industry findings into a single analytical framework, providing both **descriptive** and **predictive** perspectives on technology adoption.

Methodology

This capstone project involved a comprehensive analysis of the **Stack Overflow Developer Survey dataset**, simulating the role of an Associate Data Analyst in a technology consulting organization. The methodology outlines the process followed to extract, clean, analyze, and visualize insights from the dataset, as well as the challenges encountered along the way.

1. Data Collection

The dataset used in this project contains responses from developers worldwide regarding the technologies they currently use and wish to learn in the future. The data was accessed via the **Stack Overflow Developer Survey** and pre-processed to ensure consistency. Additional data was collected using **API access** and **web scraping techniques** to fill gaps or validate the dataset where necessary.

2. Data Cleaning and Preparation

The dataset posed several significant challenges that required extensive cleaning and preprocessing:

- **Missing values:** Key columns such as **CompTotal** (total compensation) and **ConvertedComp** had a large number of null values.
- **Incorrect formats:** Columns like **Age** and **YearsCodePro** were recorded as text instead of numeric formats, which required conversion for analysis.
- **Employment type inconsistencies:** The **EmploymentType** column contained multiple, sometimes conflicting entries (e.g., retired, student, not employed), which needed to be standardized.
- **Multiple entries in columns:** Some columns included multiple responses in a single field, requiring parsing and normalization.

Using **Python (Pandas, NumPy)**, these issues were systematically addressed. Duplicate rows were removed, missing values were handled using appropriate techniques, and data types were corrected. The final dataset was clean, consistent, and ready for exploratory analysis.

3. Exploratory Data Analysis (EDA)

After cleaning, the dataset underwent **EDA** to understand trends and distributions. This step involved calculating descriptive statistics and visualizing the data.

A particular challenge during plotting was the presence of **a large number of null values** in critical columns such as **CompTotal**, **ConvertedComp**, and **YearsCodePro**. Handling these missing values required careful consideration:

- Replacing nulls with the **mean or median** could skew the distributions, potentially affecting the accuracy of histograms and scatter plots.
- Dropping rows with missing values reduced the dataset size, which also risked distorting patterns in the visualizations.

Despite these challenges, appropriate techniques were applied to ensure the plots reasonably represented the trends in the dataset. **Histograms, scatter plots, bar charts, and pie charts** were then used to explore distributions, detect outliers, and examine correlations between variables such as programming languages, databases, and developer demographics

4. Data Visualization and Dashboard Creation

To communicate insights effectively, three interactive dashboards were created using **IBM**

Cognos Analytics and **Google Looker Studio**:

1. **Current Technology Usage Dashboard** – showing popular programming languages, databases, platforms, and frameworks.
2. **Future Technology Trends Dashboard** – highlighting technologies developers aim to learn.
3. **Demographics Dashboard** – visualizing age, employment type, location, and experience levels.

These dashboards provided clear, interactive summaries of the analysis, making the findings accessible for decision-makers and stakeholders.

5. Tools and Software

The project relied on **industry-standard tools**:

- **Python (Pandas, NumPy, Matplotlib, Seaborn)** for data cleaning, analysis, and visualization
- **SQL** for querying and managing relational data
- **Jupyter Notebook** for documenting the analysis
- **IBM Cognos Analytics / Google Looker Studio** for dashboard creation

Exploratory Data Analysis (EDA Results)

The cleaned dataset was analyzed to identify key trends in developer preferences, technology usage, and demographic patterns. EDA provided insights that formed the basis for dashboards and visual storytelling.

1. Programming Languages

- **Current Usage:** The analysis revealed that **Python, JavaScript, and SQL** are the most widely used programming languages among developers. These languages dominate due to their versatility, ease of learning, and relevance in web development, data analysis, and automation.
- **Future Interests:** Developers showed growing interest in **Go, TypeScript, and Rust**, reflecting a shift toward performance-oriented and modern development tools.

Insight: Organizations seeking to hire or train developers should prioritize skills in both dominant and emerging languages to stay competitive.

2. Databases and Platforms

- **Current Usage:** Relational databases such as **MySQL** and **PostgreSQL** remain highly popular, while **MongoDB** shows rising adoption among developers working with NoSQL solutions.
- **Future Interests:** Developers expressed interest in learning **cloud-based databases** and **distributed systems**, highlighting the industry trend toward scalable, cloud-native technologies.

3. Frameworks and Tools

- Frontend frameworks like **React** and **Angular** continue to lead in usage. Backend frameworks such as **Django** and **Node.js** are also popular among professional developers.
- Emerging tools and platforms for **machine learning and data analytics** (e.g., TensorFlow, PyTorch) are gaining interest for future learning, reflecting the shift toward AI-driven development.

4. Demographics and Developer Profiles

- **Age Distribution:** Most survey respondents were in the **25–34 age group**, representing early- to mid-career developers.
- **Experience Levels:** A significant number of developers have **5–10 years of professional coding experience**, indicating a mature and skilled workforce.
- **Employment Type:** The dataset highlighted a variety of employment types, including full-time, part-time, freelance, and student developers, providing insights into diverse workforce structures.

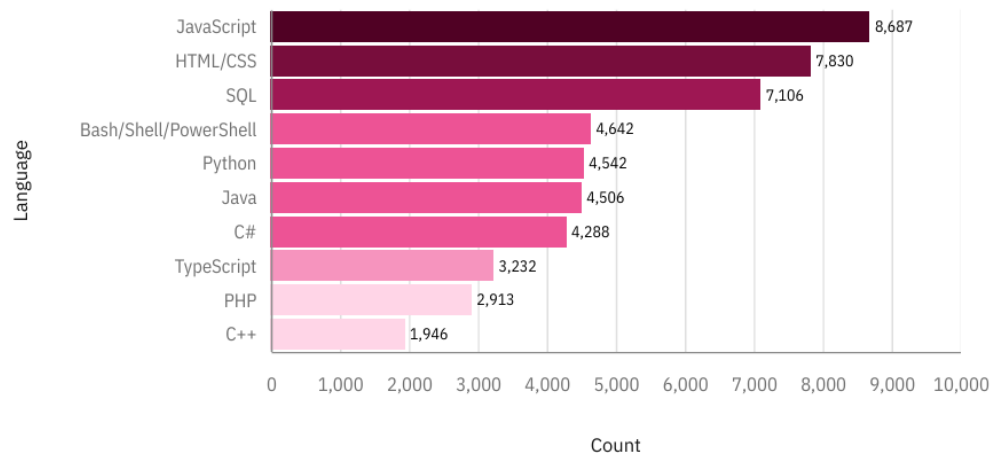
5. Challenges in Visualization

- Handling **null values and inconsistent formats** made plotting challenging. For instance, filling missing values with means or dropping rows could alter the visual trends.
- Despite these limitations, histograms, bar charts, pie charts, and scatter plots were used effectively to capture key patterns, detect outliers, and highlight relationships between variables.

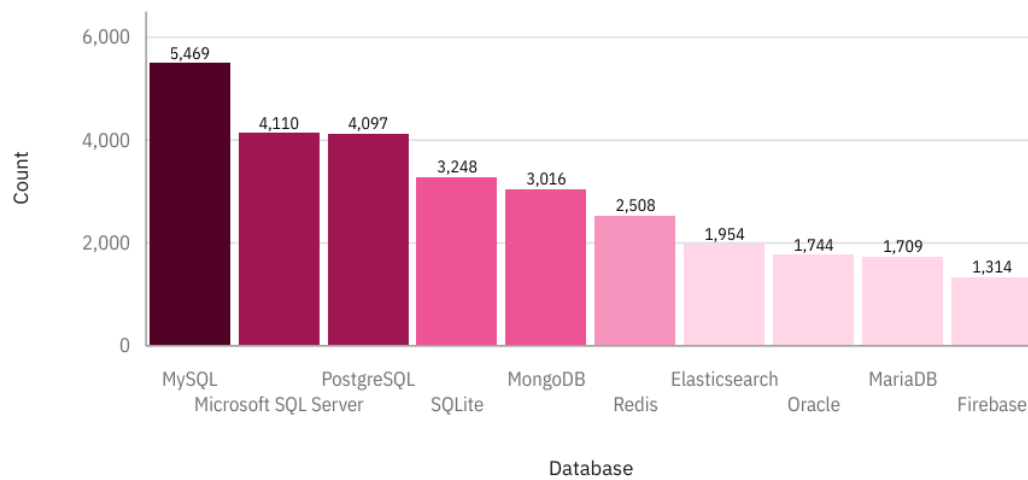
6. Summary of Key Insights

- **Dominant Technologies:** Python, JavaScript, SQL, MySQL, PostgreSQL, React, Angular
- **Emerging Trends:** Go, TypeScript, Rust, cloud databases, AI/ML tools
- **Demographics:** Majority young developers, 0 to 5 years experience, diverse employment types
- **Visualization Insight:** Careful handling of missing and inconsistent data is crucial to preserve trends in EDA and dashboard development.

Top 10 Languages Worked With



Top 10 Databases Worked With



Data Visualization and Dashboard Creation

Following the Exploratory Data Analysis (EDA), the insights were translated into **interactive dashboards** to communicate findings clearly and effectively. Visualization played a crucial role in presenting trends, relationships, and patterns in a format accessible to stakeholders, enabling informed decision-making.

1. Dashboard Overview

Three primary dashboards were developed using **IBM Cognos Analytics** and **Google Looker Studio**:

1. Current Technology Usage Dashboard

- Displays the most commonly used programming languages, databases, frameworks, and platforms.
- Includes visualizations such as **bar charts** for top languages, **pie charts** for database distribution, and **histograms** for framework usage.
- Highlights dominant tools in the developer community and identifies technologies with high adoption rates.

2. Future Technology Trends Dashboard

- Illustrates the technologies developers aim to learn in the future.
- Shows emerging trends in programming languages (e.g., Go, TypeScript, Rust), frameworks, and cloud-based tools.
- Uses **stacked bar charts** and **bubble charts** to indicate interest levels across different developer demographics.

3. Demographics Dashboard

- Provides insights into the distribution of developers by age, years of professional experience, and employment type.
- Highlights correlations between demographic factors and technology adoption patterns.
- Visualizations include **scatter plots**, **box plots**, and **histograms** to represent workforce diversity and trends.

2. Challenges in Visualization

- The dashboards required careful handling of **missing and inconsistent data** to ensure accurate visual representation.
- Columns like **CompTotal**, **ConvertedComp**, and **YearsCodePro** had numerous null values and non-numeric entries, which could distort visual trends if not processed correctly.
- Appropriate preprocessing methods, including **imputation**, **normalization**, and **filtering**, were applied to maintain the integrity of visual insights.

3. Design Considerations

- **Clarity:** All charts include descriptive titles, axis labels, and legends to enhance interpretability.
- **Color Coding:** Consistent color schemes were used to differentiate between categories and trends.
- **Interactivity:** Filters and drop-down menus were added to allow stakeholders to explore subsets of data dynamically.
- **Export and Sharing:** Dashboards were exported as PDF files for portfolio submission and sharing with peers or evaluators.

4. Key Insights from Dashboards

- **Technology Adoption:** Python, JavaScript, and SQL dominate current usage; Go, TypeScript, and Rust are the most desired languages for future learning.
- **Database Trends:** MySQL and PostgreSQL are widely used, while MongoDB and cloud-based databases are increasingly popular.
- **Demographics Influence:** Younger developers and early-career professionals show greater interest in emerging technologies, while experienced developers focus on established tools.

5. Summary

The dashboards effectively transform complex survey data into actionable insights. By combining **visual storytelling** with interactive features, stakeholders can quickly identify **current usage patterns, emerging trends, and demographic influences** in the global developer community. This approach highlights the practical value of data analytics in guiding technology strategy and workforce development.

Discussion of Insights

Programming Languages

- **Usage concentration at the top.** JavaScript (8,687), HTML/CSS (7,830), and SQL (7,106) form a clear leading tier, indicating strong emphasis on web development and data access skills.
- **Broad mid-cluster.** Bash/Shell/PowerShell (4,642), Python (4,542), Java (4,506), and C# (4,288) are closely grouped, suggesting widespread, general-purpose adoption across scripting, data, and enterprise back-end contexts.
- **Typed web development presence.** TypeScript ranks eighth (3,232), reflecting growing interest in type-safe JavaScript ecosystems.
- **Persistent legacy/systems usage.** PHP (2,913) and C++ (1,946) remain in the top ten, indicating continued maintenance and development of legacy web stacks and performance-critical software.

Implications. Training and hiring strategies that prioritize the web stack (JavaScript/HTML/CSS) and data access (SQL) will cover the largest share of respondent skills. Python, Java, and C# provide complementary depth across data science and enterprise services; TypeScript capability strengthens modern front-end reliability.

Databases

- **Relational dominance with diversification.** MySQL leads (5,469). Microsoft SQL Server (4,110) and PostgreSQL (4,097) are effectively tied for second, showing strong enterprise and open-source relational footprints.
- **Lightweight and embedded use cases.** SQLite at fourth (3,248) highlights extensive use in local, mobile, and prototyping scenarios.

- **Targeted operational data stores.** MongoDB (3,016) leads non-relational options, while Redis (2,508) and Elasticsearch (1,954) indicate prevalent caching and search/analytics patterns in production architectures.
- **Enterprise/compatibility niches.** Oracle (1,744) and MariaDB (1,709) maintain meaningful but comparatively smaller adoption; Firebase (1,314) reflects serverless/managed backend usage, especially for rapid application development.

Implications. Standardizing on a primary relational system (MySQL, SQL Server, or PostgreSQL) will align with the largest user bases. Including MongoDB, Redis, and Elasticsearch in the reference architecture supports common modern workloads (document storage, caching, search). SQLite and Firebase should be considered for edge, mobile, or quick-start projects.

Overall Recommendation.

For maximum coverage and flexibility, prioritize:

1. **Core engineering skills:** JavaScript/HTML/CSS with SQL.
2. **General-purpose back end and data:** Python or Java/C#, depending on analytics vs. enterprise needs.
3. **Data platforms:** One primary relational store plus selective adoption of MongoDB/Redis/Elasticsearch to meet document, cache, and search requirements.

Conclusion.

The survey evidence shows a concentrated skills landscape anchored by the web stack and relational databases. JavaScript (8,687), HTML/CSS (7,830), and SQL (7,106) form the leading tier of languages, with a tightly clustered second tier Bash/Shell/PowerShell (4,642), Python (4,542), Java (4,506), and C# (4,288) that spans scripting, data, and enterprise back-end use cases. TypeScript's top-ten presence (3,232) confirms growing demand for type-safe front-end ecosystems, while PHP (2,913) and C++ (1,946) underscore ongoing investment in legacy web and performance-critical systems. On the data layer, MySQL (5,469) leads, followed by a near-tie between Microsoft SQL Server (4,110) and PostgreSQL (4,097). SQLite (3,248) signals extensive local/mobile workflows; MongoDB (3,016), Redis (2,508), and Elasticsearch (1,954) indicate common document, caching, and search patterns; Oracle (1,744), MariaDB (1,709), and Firebase (1,314) round out specialized and managed-backend needs. Overall, the results point to a pragmatic technology mix: a dominant core for breadth of coverage, complemented by targeted tools for performance, scalability, and developer velocity.

Recommendations.

1. Core Skill Focus.

- *Prioritize hiring and upskilling in **JavaScript, HTML/CSS, and SQL** to maximize coverage across front-end and data-access workflows.*
- *Elevate **TypeScript** as the default for new front-end development to improve reliability and maintainability.*

2. Back-End Tracks by Business Need.

- *For analytics- and automation-heavy use cases, invest in **Python** (data tooling, ML integration).*
- *For enterprise and API-centric services, maintain strong capability in **Java** and/or **C#** with standardized frameworks.*

- Retain **Bash/Shell** proficiency for DevOps, CI/CD, and platform scripting.

3. **Data Platform Standardization.**

- Adopt a **primary relational standard** (choose one of **MySQL**, **SQL Server**, or **PostgreSQL**) based on licensing, ecosystem alignment, and existing operational expertise.
- Incorporate **MongoDB** for document models, **Redis** for caching/session/state acceleration, and **Elasticsearch** for search/log analytics, governed by explicit selection criteria (latency targets, query patterns, and cost envelopes).
- Use **SQLite** for local/mobile and prototyping scenarios; consider **Firebase** for rapid MVPs where managed services outweigh portability concerns.

4. **Architecture and Governance.**

- Define reference architectures pairing **TypeScript front-ends** with **service-oriented Java/Python back-ends** and the standardized data stack above.
- Implement **data modeling conventions** (naming, indexing, partitioning) and **observability baselines** (metrics, tracing, log search) aligned to the chosen databases.
- Establish a **technology review board** to evaluate exceptions to standards, manage deprecations (e.g., legacy PHP/C++ footprints), and ensure cost/performance accountability.

5. **Talent and Enablement.**

- Launch targeted training paths: **TypeScript migration**, **SQL performance tuning**, and **Python for data workflows**.
- Create reusable **dashboard templates** and Looker components to enforce consistency, with periodic audits for accuracy and interpretability.